

Grover's algorithm

🕒 15 mins

Category Workflow example Topics Scheduling



Background

Amplitude amplification is a general purpose quantum algorithm, or subroutine, that can be used to obtain a quadratic speedup over a handful of classical algorithms. [Grover's algorithm](#) was the first to demonstrate this speedup on unstructured search problems. Formulating a Grover's search problem requires an oracle function that marks one or more computational basis states as the states we are interested in finding, and an amplification circuit that increases the amplitude of marked states, consequently suppressing the remaining states.

Here, we demonstrate how to construct Grover oracles and use the `GroverOperator` from the Qiskit circuit library to easily set up a Grover's search instance. The runtime `Sampler` primitive allows seamless execution of Grover circuits.

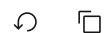
Setup

Here we import the small number of tools we need for this tutorial.



 [Authenticate to run code cells](#) [Sign in](#)

No output produced



 [Authenticate to run code cells](#) [Sign in](#)

Output:

```
'ibm_algiers'
```

Step 1: Map classical inputs to a quantum problem

Grover's algorithm requires an **oracle** that specifies one or more marked computational basis states, where "marked" means a state with a phase of -1. A controlled-Z gate, or its multi-controlled generalization over N qubits, marks the $2^N - 1$ state ('1' * N bit-string). Marking basis states with one or more '0' in the binary representation requires applying X-gates on the corresponding qubits before and after the controlled-Z gate; equivalent to having an open-control on that qubit. In the following code, we define an oracle that does just that, marking one or more input basis states defined through their bit-string representation. The `MCMT` gate is used to implement the multi-controlled Z-gate.



 [Authenticate to run code cells](#) [Sign in](#)

No output produced

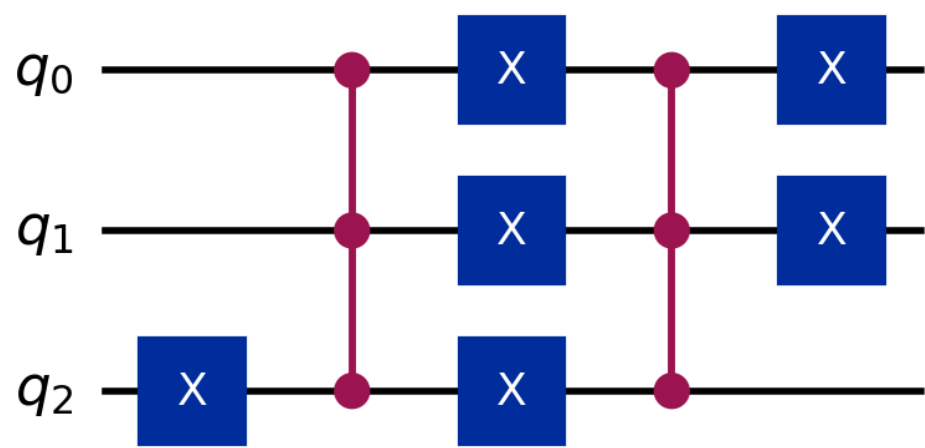
Specific Grover's instance

Now that we have the oracle function, we can define a specific instance of Grover search. In this example we will mark two computational states out of the eight available in a three-qubit computational space:



 [Authenticate to run code cells](#) [Sign in](#)

Output:



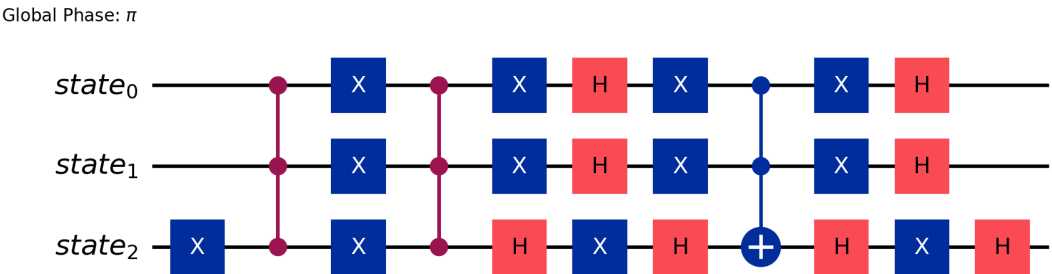
GroverOperator

The built-in Qiskit `GroverOperator` takes an oracle circuit and returns a circuit that is composed of the oracle circuit itself and a circuit that amplifies the states marked by the oracle. Here, we `decompose` the circuit to see the gates within the operator:



 [Authenticate to run code cells](#) [Sign in](#)

Output:



Repeated applications of this `grover_op` circuit amplify the marked states, making them the most probable bit-strings in the output distribution from the circuit. There is an optimal number of such

applications that is determined by the ratio of marked states to total number of possible computational states:



[Authenticate to run code cells](#) [Sign in](#)

No output produced

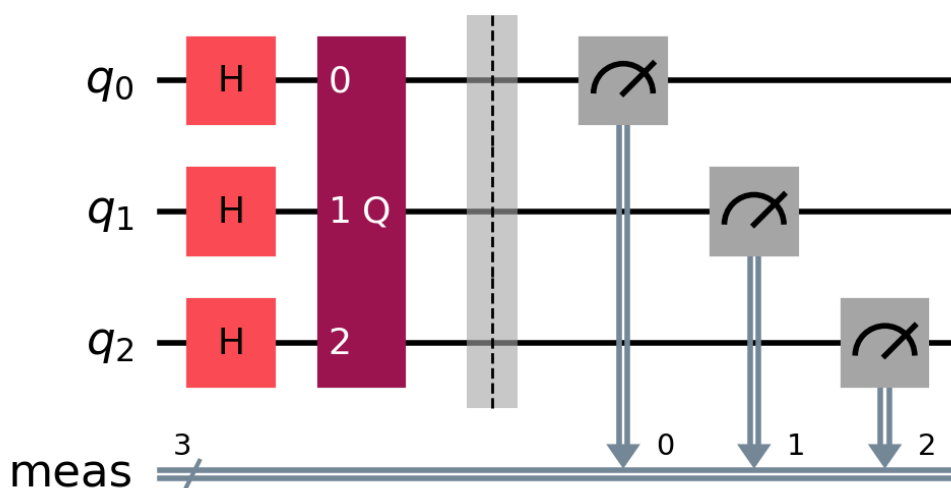
Full Grover circuit

A complete Grover experiment starts with a Hadamard gate on each qubit; creating an even superposition of all computational basis states, followed the Grover operator (`grover_op`) repeated the optimal number of times. Here we make use of the `QuantumCircuit.power(INT)` method to repeatedly apply the Grover operator.



[Authenticate to run code cells](#) [Sign in](#)

Output:

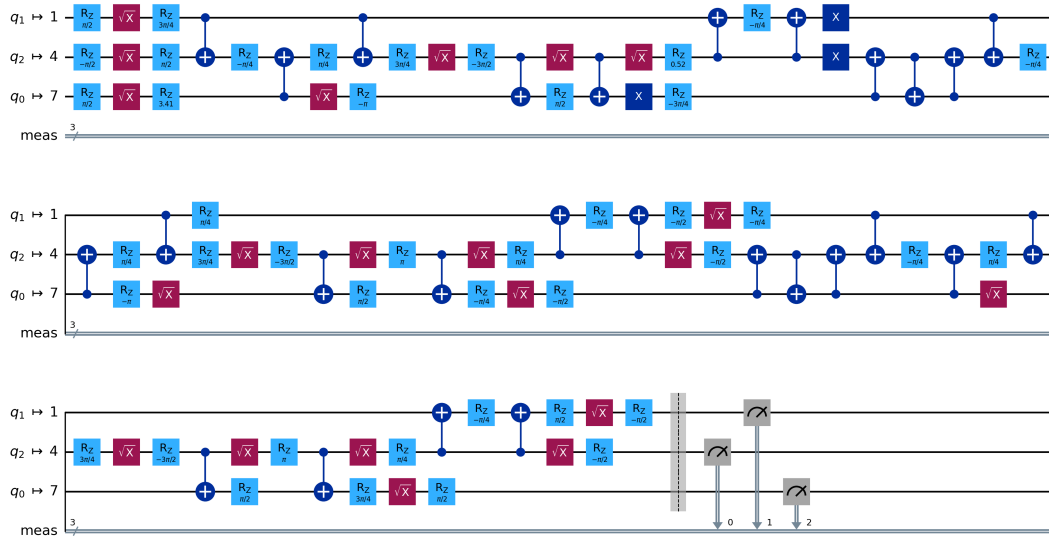


Step 2: Optimize problem for quantum execution.



[Authenticate to run code cells](#) [Sign in](#)

Output:



Step 3: Execute using Qiskit Primitives.

Amplitude amplification is a sampling problem that is suitable for execution with the `Sampler` runtime primitive.

Note that the `run()` method of [Qiskit Runtime SamplerV2](#) takes an iterable of primitive unified blocs (PUBs). For sampler, each PUB is an iterable in the format `(circuit, parameter_values)`. However, at a minimum, it takes a list of quantum circuit(s).



[Authenticate to run code cells](#) [Sign in](#)

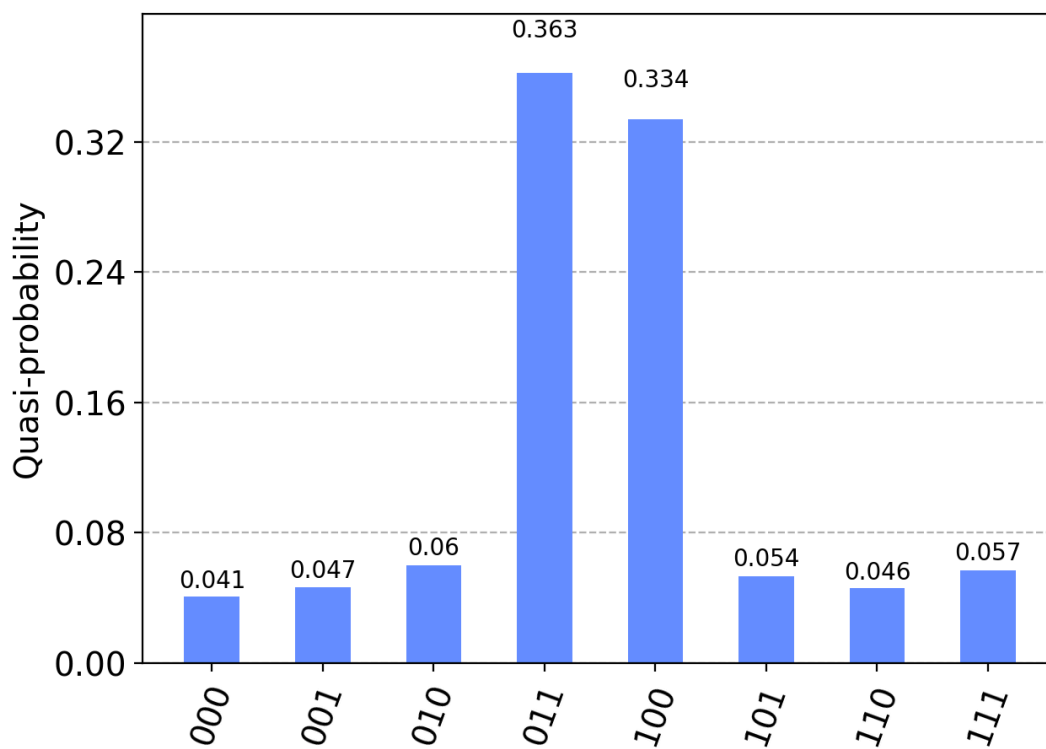
No output produced

Step 4: Post-process, return result in classical format.



 [Authenticate to run code cells](#) [Sign in](#)

Output:



 [Authenticate to run code cells](#) [Sign in](#)

Output:

'0.21.1'